

LogicConnect: Structured Connectivity Design for Differentiable Logic Gate Networks

Abstract—Differentiable logic gate networks learn Boolean circuits whose connectivity determines which signals can be combined during gate-function optimization. Learning these connections can improve accuracy, but increases the memory required to process candidate inputs during training. In this paper, we present LogicConnect, a structured connectivity method that constructs the graph before training while preserving the network’s gate budget. LogicConnect combines semantic input pairing with regular multiscale matchings in deeper layers and supports dense and convolutional architectures without trainable routing parameters. We evaluate the method on MNIST, Fashion-MNIST, and CIFAR-10 through accuracy comparisons, training-resource measurements, and structural ablations. LogicConnect improves dense CIFAR-10 test accuracy by 3.18–4.59 percentage points across three model sizes and three paired seeds. Convolutional S/M models improve by 3.26/2.08 points in single-seed tests, although the three-seed S validation comparison remains inconclusive. Compared with learned Top-32 routing, LogicConnect uses 93.7–94.2% less peak GPU memory and 80% fewer trainable parameters, with an accuracy reduction of 1.84–5.30 points. Degree-preserving ablations identify first-layer pairing as the strongest measured dense contribution, supporting structured connectivity as a design choice for improving accuracy under constrained training memory.

I. INTRODUCTION

Differentiable Logic Gate Networks (DLGNs) provide an alternative path toward efficient machine learning at the edge by learning directly with the primitive operations used in digital hardware. Rather than relying on multiply-accumulate operations, a DLGN is composed of Boolean gates. During training, these discrete operations are represented through differentiable relaxations, allowing the network to be optimized with gradient descent. After training, each relaxed gate is hardened to a Boolean function for efficient inference [1]. This direct correspondence between the learned model and Boolean computation makes DLGNs particularly attractive for resource-constrained embedded and edge and systems.

However, a fundamental characteristic of DLGNs is their sparse connectivity. Each conventional DLGN gate has only two inputs. These two predecessors are selected before training and remain fixed, while optimization determines which Boolean function the gate should implement [1]. Therefore, training can change *how* two signals are combined, but not *which* signals arrive at a gate. To that end, if two informative signals are never connected through the available graph, learning the gate

functions alone cannot create that missing edge. At a fixed depth and width, connectivity consequently determines the combinations of information that can be formed throughout the network and becomes part of the effective architectural capacity of the DLGN.

A way to address this issue is to consider connectivity as an independent design dimension [2] and optimize the predecessors themselves during training [3], [4]. This type of learned connectivity can recover connections that were absent from the original graph and can therefore improve accuracy. However, such methods require significantly increased training time as each gate must consider multiple candidate signals, requiring additional routing state and candidate activations even though the final hardened network may retain only a small number of predecessors. This creates a practical design choice between inexpensive but arbitrary *fixed random connectivity* and more flexible but substantially heavier *learned connectivity*.

To demonstrate the need for connectivity design, we consider a motivational example on a dense CIFAR-10 DLGN. We keep the deeper connectivity unchanged and randomize only the first-layer connections under a degree-preserving constraint. Hence, every predecessor is used exactly the same number of times before and after randomization; only the identities of the signals paired at each gate are changed. Despite preserving the same fan-out, hardened validation accuracy decreases by 4.49 percentage points. This result reveals an important limitation of degree-balanced connectivity: *how often a signal is used does not determine which information is brought together*. However, explicitly selecting favorable pairs is itself challenging to scale. Given N candidate predecessors, a two-input gate can already choose from $O(N^2)$ possible pairs, and this choice is repeated across thousands or millions of gates and multiple network layers. Moreover, while input signals have explicit image semantics, intermediate signals progressively lose this direct interpretation, making a manually defined pairing rule increasingly difficult to justify at deeper layers. Fixed random connectivity avoids this search but ignores signal relationships, whereas learned connectivity explores the enlarged space by maintaining additional routing state and candidate activations during training. Therefore, a key challenge is to construct informative connections *before training* using principles that remain scalable as the network grows and do not introduce the training overhead of a separate routing problem.

In this paper, we propose **LogicConnect**, an offline connectivity construction that introduces structure into the DLGN graph before training. LogicConnect first exploits signal identity where it is most explicit by forming semantic first-layer pairs

according to relationships among the encoded inputs rather than arbitrary indices. In subsequent layers, it organizes predecessor interactions through regular multiscale matchings, selecting complete matching stages primarily to balance predecessor usage and using ancestry overlap to distinguish alternatives with equivalent degree balance.

II. RELATED WORK

Petersen et al. introduced Differentiable Logic Gate Networks (DLGNs), enabling gradient-based optimization of Boolean gate functions while keeping randomly initialized connections fixed during training [1]. In [5], the authors extended DLGNs to convolutional architectures, exploiting image structure through shared logic trees, spatial sampling, pooling, and residual initialization. Other works extend DLGNs to recurrent models [6], improve output aggregation for larger classification problems [7], or develop interpretable and truth-table-based logic architectures [8]–[10]. Related differentiable logic models have also been explored for cellular automata [?], [11] and circuit representation learning [12]. However, these works primarily focus on extending the architecture and applicability of logic-based networks, without exploring the connectivity problem.

Regarding DLGN connectivity, Mommen et al. optimize distributions over candidate predecessors, allowing gate inputs to change during training [3]. Similarly, LILogic Net employs sparse Top-K routing to learn compact network connectivity [4], while BitLogic studies connectivity together with encoding, fan-in, node parameterization, and output-head design [2]. These works demonstrate that selecting better predecessors can significantly improve logic networks. However, learned routing requires processing candidate connections and maintaining additional routing parameters and intermediate activations during training, increasing both execution time and memory requirements, even though the hardened network ultimately retains only a small number of inputs per gate. In contrast, our approach constructs the connectivity *offline* and keeps it fixed during gate training, avoiding an additional routing optimization problem while preserving the declared depth, width, and gate budget.

Additionally, several works focus on improving the optimization of the gate functions themselves. Mind the Gap reduces the discrepancy between continuous training and discrete inference [?]. Light DLGNs redesign the gate parameterization to improve training efficiency [13], while WARP introduces a Walsh-based Boolean representation [14]. The works in [15], [16] further explore polynomial formulations to analyze or replace conventional soft gate mixtures. These methods optimize *what function* a gate computes from its assigned inputs, whereas our work focuses on *which signals* are made available to that gate.

III. METHODOLOGY

LogicConnect constructs the connectivity of a DLGN before training, with the goal of exposing each gate to more informative combinations of signals while retaining the simplicity of fixed connectivity. As illustrated in Figure ??, LogicConnect takes the encoded input representation and the dimensions of each network layer and returns the two fixed predecessor indices required by every gate. Once these connections are generated, the graph

is frozen and training proceeds using the original DLGN gate parameterization. Therefore, LogicConnect does not introduce additional routing parameters, change the number of gates, or modify the gate-training procedure.

The key challenge is that the information available for connectivity design changes with network depth. At the network input, signals retain explicit relationships inherited from the data representation, such as their spatial location, channel, and encoding. LogicConnect exploits these relationships through *semantic input pairing*, avoiding an arbitrary selection of inputs. After signals pass through logic gates, however, this direct interpretation is progressively lost. Rather than imposing the same semantic rule at every layer, LogicConnect organizes subsequent connections through *degree-balanced multiscale matchings*. These matchings systematically combine predecessors while controlling their reuse based on structural ancestry.

Importantly, the construction is entirely architecture-driven. LogicConnect does not observe class labels, training loss, validation or test accuracy, or trained gate parameters when selecting connections. It also does not search over individual edges using accuracy feedback or perform post-hoc edge swapping. Instead, it selects from predefined structured pairing and matching stages using deterministic structural criteria. Thus, the one-time construction is separated from learning: connectivity is designed offline, while conventional DLGN training subsequently determines only the Boolean functions implemented by the fixed graph.

A. Fixed-Graph Representation

We first define the fixed connectivity used throughout the network. For a layer l with m_l two-input gates and n_l available predecessors, we represent its connections by $I_l \in \{0, \dots, n_l - 1\}^{2 \times m_l}$. For every gate j , the two entries $I_l[0, j]$ and $I_l[1, j]$ identify distinct predecessor signals. Following the original DLGN formulation [1], the output of gate j during training is computed as

$$h_{l,j} = \sum_{q=0}^{15} \text{softmax}(\theta_{l,j})_q \tilde{f}_q(h_{l-1,I_l[0,j]}, h_{l-1,I_l[1,j]}), \quad (1)$$

where $\theta_{l,j}$ contains the trainable logits over the 16 two-input Boolean functions and $\tilde{f}_q$ denotes their continuous relaxation. After training, the function with the highest weight is selected for discrete inference. LogicConnect modifies only the fixed predecessor indices I_l ; the gate parameterization, optimization objective, discretization procedure, and output aggregation remain unchanged. This allows us to isolate the effect of connectivity from changes to the DLGN training formulation.

To preserve information about how signals are related as they propagate through the fixed graph, we associate every predecessor with a *structural ancestry set*. At the network input, each encoded signal is assigned a source identity describing the raw value from which it originates. For image inputs, for example, multiple thermometer-encoded thresholds of the same pixel and channel share the same source identity. The ancestry of

an input therefore contains its source label. For a gate receiving predecessors a and b , its ancestry is propagated simply as

$$A_{l,j} = A_{l-1,a} \cup A_{l-1,b}. \quad (2)$$

Importantly, $A_{l,j}$ describes the set of sources that are *structurally reachable* by the gate, rather than the inputs that functionally determine its output. The learned Boolean function may suppress or ignore one of its predecessors. We use this structural information only during offline connectivity construction; it does not introduce trainable parameters or alter the gate computation.

B. Semantic Input Pairing

The first layer is the only part of the network where the identity of each signal is known directly from the input representation. Rather than treating these inputs as arbitrary bit indices, we use their source metadata to expose the gates to relationships already present in the encoded image. We denote an encoded input by $x_{p,c,t}$, where p identifies the spatial position, c the channel, and t the encoding threshold. All thresholds originating from the same raw pixel-channel value share the same source identity, $\sigma(x_{p,c,t}) = (p, c)$. Therefore, pairs satisfying $\sigma(x_i) = \sigma(x_j)$ are excluded, since they originate from the same underlying image value and provide limited complementary information.

Using this metadata, we construct a collection of semantic pairing stages. Spatial stages connect inputs whose positions differ by power-of-two displacements,

$$\Delta = \{1, 2, 4, \dots\}, \quad (3)$$

up to half of the corresponding image dimension. Horizontal and vertical displacements are considered separately, with cyclic boundaries used to avoid systematically favoring particular image regions. Additional stages capture cross-channel and cross-threshold relationships, including aligned channels and threshold combinations with spatial displacement. Hence, the candidate set can be viewed as

$$C = C_{\text{spatial}} \cup C_{\text{channel}} \cup C_{\text{threshold}}, \quad (4)$$

where every pair in C satisfies the distinct-source constraint above. This exposes the first-layer gates to multiple complementary notions of input structure instead of relying on a single local pairing rule.

Semantic structure alone, however, does not guarantee balanced predecessor usage. Let d_i denote the number of times predecessor i has already been selected. For a candidate pair (i, j) , LogicConnect prioritizes smaller

$$d_i + d_j, \quad (5)$$

followed by the smaller value of $\max(d_i, d_j)$, and finally the pair indices to resolve remaining ties. This degree-aware ordering prevents a small subset of semantically favorable inputs from dominating the first layer. Candidate stages are visited cyclically so that spatial, channel, and encoding relationships all contribute to the final graph.

To keep the construction scalable, we avoid searching the complete $\mathcal{O}(n^2)$ space of possible predecessor pairs. At each selection, the constructor examines at most 64 unused candidates

following a fixed seeded affine traversal and applies the degree criteria above. Once the distinct pairs of a stage are exhausted, repeated pairs are allowed under the same ordering. Importantly, this bounded search is purely structural: class labels, training loss, validation accuracy, and trained gate parameters are never used to select a pair. The first-layer connectivity is therefore fully determined before training from the input metadata and the requested network dimensions.

C. Degree-Balanced Multiscale Matching

Semantic pairing is well-defined at the network input because every signal retains explicit spatial, channel, and encoding metadata. This interpretation does not persist in deeper layers, where each predecessor represents the output of several previously combined logic gates. Reapplying the same input-level semantic rules would therefore impose relationships that are no longer directly supported by the signal representation. Instead, LogicConnect uses a regular *multiscale matching* construction that progressively mixes hidden signals while controlling how frequently each predecessor is reused.

For a layer with n available predecessors, we consider $K = \lceil \log_2 n \rceil$ nominal interaction scales. When n is a power of two, scale s forms the matching

$$\mathcal{M}_s = \{(i, i \oplus 2^s)\}, \quad s = 0, \dots, \log_2 n - 1, \quad (6)$$

where each unordered pair is retained only once. Consequently, every stage contains $n/2$ disjoint pairs and every predecessor appears exactly once. Different values of s expose the gates to progressively different index separations. For example, with eight predecessors,

$$\begin{aligned} s = 0 &: (0, 1), (2, 3), (4, 5), (6, 7), \\ s = 1 &: (0, 2), (1, 3), (4, 6), (5, 7), \\ s = 2 &: (0, 4), (1, 5), (2, 6), (3, 7). \end{aligned}$$

This structure provides regular mixing at multiple scales without searching over arbitrary edge combinations.

For widths that are not powers of two, we preserve the same principle using affine traversals of the predecessor indices. A traversal follows $\pi(k) = (o + kr) \bmod n$, where the step r is chosen to be coprime with n , and adjacent elements of the traversal are paired. This guarantees that the traversal visits every predecessor before repeating. For odd n , one predecessor necessarily remains unmatched in each complete stage; the unmatched position is rotated across stages to avoid systematically excluding the same signal. Thus, a complete matching contains $\lfloor n/2 \rfloor$ pairs. If fewer gates remain than required for a complete stage, LogicConnect uses a final partial matching and selects disjoint pairs according to endpoint degree sum, maximum endpoint degree, and index order.

The remaining question is *which matching stage should be used next*. LogicConnect follows a strict selection hierarchy. First, it prioritizes balanced predecessor reuse. Let d_i denote the current number of times predecessor i has been selected, and let $\delta_i(\mathcal{M}_s)$ denote its additional occurrences if stage s is appended. We measure the resulting degree imbalance as

$$D_s = \max_i (d_i + \delta_i(\mathcal{M}_s)) - \min_i (d_i + \delta_i(\mathcal{M}_s)). \quad (7)$$

Stages with smaller D_s are preferred. This criterion prevents repeated selection of a small subset of predecessors and keeps access to the preceding layer as uniform as possible.

Degree balance alone, however, cannot distinguish stages that use predecessors equally often but repeatedly combine signals with similar structural history. We therefore use the ancestry sets introduced in Section III-A only as a secondary tie-breaking criterion. For a candidate stage, we define its ancestry novelty as

$$N_s = \frac{1}{|Q_s|} \sum_{(a,b) \in Q_s} \left(1 - \frac{|A_a \cap A_b|}{\max(1, \min(|A_a|, |A_b|))} \right), \quad (8)$$

where Q_s contains all pairs of the candidate matching, or at most 2,048 evenly sampled pairs for very large stages. Identical nonempty ancestry sets contribute zero, whereas disjoint sets contribute one. Hence, among stages with the same degree spread, LogicConnect favors the one that combines predecessors with less redundant reachable support. The ancestry score does not override degree balance; it is used only when the primary criterion cannot distinguish between candidate stages.

Any remaining ties are resolved by a deterministic cyclic scale order, with every nominal scale used once before the cycle restarts. Importantly, LogicConnect selects *whole predefined matching stages*. It does not optimize individual edges, swap connections according to validation accuracy, or perform an accuracy-driven search over candidate graphs. Only the final truncated stage, required when the remaining number of gates is smaller than a complete matching, is selected at the pair level. This stage-based construction substantially reduces the connectivity search space while retaining controlled predecessor reuse and multiscale signal mixing.

Algorithm 1 summarizes the hidden-layer construction.

D. Convolutional DLGNs

The same connectivity principles can be applied to convolutional DLGNs, but the object being connected changes from individual signals to shared channel groups. Convolutional DLGNs already impose a strong spatial inductive bias through their receptive-field sampler and shared logic trees [5]. LogicConnect therefore does not alter the spatial sampling pattern or replace the convolutional computation. Instead, it constructs the *channel pairs* presented to the shared logic trees, allowing connectivity to be structured while preserving the original convolutional execution model.

More specifically, the receptive-field sampler, tree depth, gate sharing, and pooling operations remain identical to the baseline convolutional DLGN. For the first convolutional layer, channel assignments are constructed using the available input metadata, analogous to the semantic pairing described in Section III-B. Subsequent channel groups are formed using the degree-balanced multiscale matching of Section III-C. Since the same channel pairing is reused across spatial positions, the construction preserves weight sharing and does not introduce location-dependent routing. Thus, LogicConnect changes which channels are combined, while the spatial locations sampled by the convolutional operator remain unchanged.

To support the same structural criteria across convolutional layers, ancestry is tracked at the channel level. Initially, each

Algorithm 1 Degree-Balanced Multiscale Matching

Require: Number of predecessors n , number of gates m , ancestry sets $\{A_i\}$

Ensure: Fixed predecessor pairs for the layer

- 1: Generate $K = \lceil \log_2 n \rceil$ multiscale candidate matchings
 - 2: Initialize predecessor degrees $d_i \leftarrow 0$
 - 3: Initialize cyclic scale order
 - 4: **while** unassigned gates remain **do**
 - 5: Determine whether a complete or partial stage is required
 - 6: **for** each eligible candidate stage s **do**
 - 7: Compute degree spread D_s
 - 8: **end for**
 - 9: Retain stages with minimum D_s
 - 10: **if** multiple stages remain **then**
 - 11: Compute ancestry novelty N_s for the tied stages
 - 12: Retain stages with maximum N_s
 - 13: **end if**
 - 14: Resolve any remaining tie using the cyclic scale order
 - 15: Append the selected complete or partial matching
 - 16: Update predecessor degrees and ancestry
 - 17: **end while**
 - 18: **return** Fixed predecessor pairs
-

encoded input channel is assigned a source label. After a shared logic tree produces a new output channel, its ancestry is formed from the union of the ancestries of its input channels. We additionally retain an identity label for each generated output channel. This distinction is important when two channels obtain identical upstream support: although their reachable sources are the same, they remain separate signals and may learn different Boolean functions. As in the dense case, these labels are used only during offline graph construction and do not become trainable features.

After the convolutional blocks, the resulting feature maps are flattened for classification. Each classifier input retains its channel ancestry together with its spatial-feature identity, allowing the fixed classifier graph to be constructed using the same source-aware pairing and multiscale matching principles introduced above. Therefore, the convolutional implementation does not require a separate connectivity-learning mechanism. LogicConnect modifies only the fixed predecessor assignments; the convolutional sampler, shared logic trees, pooling, gate parameterization, training objective, and discretization procedure remain unchanged from the underlying convolutional DLGN.

REFERENCES

- [1] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen, "Deep differentiable logic gate networks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 2006–2018, 2022.
- [2] S. Bühner, A. Plesner, T. Aczel, and R. Wattenhofer, "BitLogic: A framework for gradient-based LUT-native neural networks," *Transactions on Machine Learning Research*, Jun. 2026.
- [3] W. Mommen, L. Keuninckx, M. Hartmann, and P. Wambacq, "A method for optimizing connections in differentiable logic gate networks," *arXiv preprint arXiv:2507.06173*, 2025.
- [4] K. Fojcik, R. Zioma, and J. Armitis, "Lillogic net: Compact logic gate networks with learnable connectivity for efficient hardware deployment," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 3498–3507.
- [5] F. Petersen, H. Kuehne, C. Borgelt, J. Welzel, and S. Ermon, "Convolutional differentiable logic gate networks," *Advances in neural information processing systems*, vol. 37, pp. 121 185–121 203, 2024.
- [6] S. Bühner, A. Plesner, T. Aczel, and R. Wattenhofer, "Recurrent deep differentiable logic gate networks," in *Proceedings of the 2nd International Workshop on Edge and Mobile Foundation Models*, ser. EdgeFM '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 31–36.
- [7] S. Brändle, T. Aczel, A. Plesner, and R. Wattenhofer, "From mnist to imagenet: Understanding the scalability boundaries of differentiable logic gate networks," 2025.
- [8] C. Yue and N. K. Jha, "Learning interpretable differentiable logic networks," *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, vol. 1, no. 1, pp. 69–82, 2024.
- [9] A. Benamira, T. Guérard, T. Peyrin, T. Yap, and B. Hooi, "A scalable, interpretable, verifiable & differentiable logic gate convolutional neural network architecture from truth tables," 2023.
- [10] A. Benamira, T. Peyrin, T. Yap, T. Guérard, and B. Hooi, "Truth table net: Scalable, compact & verifiable neural networks with a dual convolutional small boolean circuit networks form." in *International Joint Conference on Artificial Intelligence (IJCAI)*, 2024, pp. 13–21.
- [11] P. Miotti, E. Niklasson, E. Randazzo, and A. Mordvintsev, "Differentiable logic cellular automata: From game of life to pattern generation," in *Artificial Life Conference Proceedings 37*, vol. 2025, no. 1. MIT Press One Rogers Street, Cambridge, MA 02142-1209, USA journals-info . . . , 2025, p. 54.
- [12] M. Li, S. Khan, Z. Shi, N. Wang, H. Yu, and Q. Xu, "Deepgate: learning neural representations of logic gates," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC '22. New York, NY, USA: Association for Computing Machinery, 2022, pp. 667–672.
- [13] L. Rüttgers, T. Aczel, A. Plesner, and R. Wattenhofer, "Light differentiable logic gate networks," in *International Conference on Learning Representations*, C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust, Eds., vol. 2026, 2026, pp. 66 744–66 776.
- [14] L. Gerlach, T. Gerlach, L. Våge, E. Kauffman, and I. Ojalvo, "Warp logic neural networks," 2026.
- [15] Y. Kim, "Fitting multilinear polynomials for logic gate networks," 2026.
- [16] S. S. Damera, R. Matheu, A. Puranic, and J. S. Baras, "Polynomial surrogate training for differentiable ternary logic gate networks," *ArXiv*, vol. abs/2603.00302, 2026.